

Báo cáo bài tập EX15: Thực hành triển khai Transformer

```
[1]: import torch.nn as nn
import torch
import torchtext
import copy
import math
import torch.nn.functional as F

[2]: device = 'cuda' if torch.cuda.is_available() else 'cpu'
device

[2]: 'cpu'

[3]: #! python -m spacy download en_core_web_sm
#! python -m spacy download fr_core_news_sm

[4]: # embedding
class Embedder(nn.Module):
    def __init__(self, vocab_size, dim): # cần kích thước của từ điển và dim của embedd
        super(Embedder, self).__init__()
        self.embedding = nn.Embedding(vocab_size, dim)
    def forward(self, X):
        return self.embedding(X)
# shape là batch_size * seq_len * dim
```

```
[4]: # embedding
class Embedder(nn.Module):
    def __init__(self, vocab_size, dim): # cần kích thước của từ điển và dim của embedd
        super(Embedder, self).__init__()
        self.embedding = nn.Embedding(vocab_size, dim)
    def forward(self, X):
        return self.embedding(X)
# shape là batch_size * seq_len * dim
```

```
[5]: # pe/ positional embedding
class PositionalEncoder(nn.Module):
    def __init__(self, dim, max_seq_len = 300): # cần dim (cùng dim với embed) và độ dài tối đa của câu (i max)
        super(PositionalEncoder, self).__init__()
        self.dim = dim
        pe = torch.zeros(max_seq_len, dim)
        for row in range(pe.size(0)):
            for col in range(pe.size(1)):
                if col%2 == 0 :
                    pe[row][col] = torch.sin(torch.tensor(row/(10000**(2*(col/2)/pe.size(1)))))
                else:
                    pe[row][col] = torch.cos(torch.tensor(row/(10000**((2*(col-1)/2)/pe.size(1)))))
        pe = pe.unsqueeze(0)
        self.register_buffer('pe', pe) # đăng kí cho nó làm buffer, có khả năng được chuyển đi cùng với model lên các thiết bị
# nếu không để là buffer thì nó sẽ không được quan tâm và ở lại cpu mãi mãi khi model được chuyển sang thiết bị khác
# có 2 tính năng 1 là tạo ra biến pe mới là thuộc tính của instance , 2 là tính năng kế trên
    def forward(self, X):
        # cần làm cho giá trị embed tương đối lớn hơn giá trị của pe bằng cách nhân với căn của dim
        X = X * math.sqrt(self.dim)
        seq_len = X.size(1)
        X = X + self.pe[:, :seq_len, :]
        return X
```

```
pe = PositionalEncoder(256)
```

```
pe.pe
```

```
tensor([[[[ 0.0000e+00,  1.0000e+00,  0.0000e+00, ...,  1.0000e+00,
            0.0000e+00,  1.0000e+00],
          [ 8.4147e-01,  5.4030e-01,  8.0196e-01, ...,  1.0000e+00,
            1.0746e-04,  1.0000e+00],
          [ 9.0930e-01, -4.1615e-01,  9.5814e-01, ...,  1.0000e+00,
            2.1492e-04,  1.0000e+00],
          ...,
          [ 9.9287e-01, -1.1921e-01, -8.0185e-02, ...,  9.9941e-01,
            3.1910e-02,  9.9949e-01],
          [ 4.3614e-01, -8.9988e-01,  7.5148e-01, ...,  9.9941e-01,
            3.2018e-02,  9.9949e-01],
          [-5.2158e-01, -8.5320e-01,  9.7801e-01, ...,  9.9940e-01,
            3.2125e-02,  9.9948e-01]]]])
```

Khi d lớn thì dot product cũng lớn theo vì thế người ta chuẩn hóa bằng cách chia cho căn của số chiều trước khi cho vào softmax

đối với các mask cần che thì có cách là chuyển chúng thành âm vô cùng vì khi đó giá trị của softmax tiến gần về 0

```
# q, k, v ở đây chính là kết quả của X nhân với các ma trận trọng số Q, K, V rồi
# vì k = (x*K) cho nên transpose của k là Kt * Xt, vì thế phải lấy q nhân với transpose của k
def attention(q, k, v, d_k, mask = None, dropout = None):
    scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k)
    if mask is not None:
        mask = mask.unsqueeze(1)
        scores = scores.masked_fill(mask == 0, -1e9) # khi mask bằng 0 thì chuyển giá trị tại đó thành -1e9
    scores = F.softmax(scores, dim = -1) # scores có shape là T*d
    if dropout is not None:
        scores = dropout(scores)
    output = torch.matmul(scores, v)
    return output
```

Đối với multihead ta cần có h ma trận Q, K, V có chiều là d*(d/h). trước tiên ta cứ concat hết chúng với nhau sẽ thành các ma trận lớn Q, K, V với chiều là d*d sau đó chúng ta nhân với X để ra các ma trận q, k, v cuối cùng lúc đó ta mới chia ra làm h phần dựa trên heads sau, có thể dùng reshape

```
class MultiHeadAttention(nn.Module):
    def __init__(self, heads, dim, dropout = 0.1):
        super().__init__()
        self.dim = dim
        self.dim_head = int(dim // heads) # Ensure integer division
        self.h = heads
        self.q_linear = nn.Linear(dim, dim)
        self.k_linear = nn.Linear(dim, dim)
        self.v_linear = nn.Linear(dim, dim)
        self.dropout = nn.Dropout(dropout)
        self.out = nn.Linear(dim, dim)

    # q, k, v lúc này chưa được nhân với các ma trận trọng số chính là đầu vào x
    # viết như vậy để tái sử dụng cho cả self attention và cross attention
    def forward(self, q, k, v, mask = None):
        bs = q.size(0) # số lượng sample trong x, batch size
        k = self.k_linear(k).view(bs, -1, self.h, self.dim_head)
        q = self.q_linear(q).view(bs, -1, self.h, self.dim_head)
        v = self.v_linear(v).view(bs, -1, self.h, self.dim_head)
        # cần transpose để thành dạng batch_size * h * sequence_len * dim_h, nếu không transpose mà cho vào hàm attention thì nó sẽ hiểu là h*dim và sẽ
        k = k.transpose(1, 2)
        q = q.transpose(1, 2)
        v = v.transpose(1, 2)
        scores = attention(q, k, v, self.dim_head, mask, self.dropout)
        # tính xong scores rồi thì transpose lại để thành dạng batch_size * sequence_len * h * dim_h sau đó gộp các head lại thành batch*sl*dim
        # tuy nhiên khi sử dụng transpose thì nó chỉ thay đổi index trên màn hình chứ không thay đổi bản thân trong ram vì thế phải sử dụng contiguous (view yêu cầu)
        concat = scores.transpose(1, 2).contiguous().view(bs, -1, self.dim)
        # khi ta concat các head lại với nhau thì chỉ là đơn thuần phép nối, các head vẫn không có thông tin gì với nhau độc lập với nhau
        # mỗi head đứng 1 vị trí riêng, thêm lớp tuyến tính để trộn các head với nhau
        output = self.out(concat)
        return output
```

```

class FeedForward(nn.Module):
    def __init__(self, d_model, d_ff = 2048, dropout = 0.1):
        # d_model là chiều của vector embedding, d_ff là chiều của lớp feed
        super(FeedForward, self).__init__()
        if d_ff is None:
            d_ff = min(d_model * 4, 2048) # Scale with d_model, max 2048
        self.linear_1 = nn.Linear(d_model, d_ff)
        self.dropout = nn.Dropout(dropout)
        self.linear_2 = nn.Linear(d_ff, d_model)

    def forward(self, X):
        X = self.linear_1(X)
        X = self.dropout(X)
        X = self.linear_2(X)
        return X

class Norm(nn.Module):
    def __init__(self, d_model, eps = 1e-6):
        super(Norm, self).__init__()
        self.size = d_model
        self.alpha = nn.Parameter(torch.ones(self.size))
        self.bias = nn.Parameter(torch.zeros(self.size))
        self.eps = eps

    def forward(self, X):
        norm = self.alpha * (X - X.mean(dim = -1, keepdim = True)) / (X.std(dim = -1, keepdim = True, unbiased=False) + self.eps) + self.bias
        # giữ lại chiều cuối cùng để có thể thực hiện được phép trừ
        return norm

```

xây dựng 1 lớp encoder

1 lớp encoder là tổ hợp của 1 multihead và 1 feed forward

```

class EncoderLayer(nn.Module):
    def __init__(self, d_model, heads, dropout = 0.1):
        super().__init__()
        self.norm1 = Norm(d_model)
        self.norm2 = Norm(d_model)
        self.attn = MultiHeadAttention(heads, d_model)
        self.ff = FeedForward(d_model)
        self.dropout_1 = nn.Dropout(dropout)
        self.dropout_2 = nn.Dropout(dropout)

    def forward(self, X, mask):
        X1 = self.dropout_1(self.attn(X,X,X,mask))
        X = self.norm1(X + X1) # add and norm
        X1 = self.dropout_2(self.ff(X))
        X = self.norm2(X + X1)
        return X

```

```
def get_clones(module, N):  
    return nn.ModuleList([copy.deepcopy(module) for i in range(N)])  
# tạo các module có architecture giống nhau nhưng trọng số khác nhau
```

```
class Encoder(nn.Module):  
    def __init__(self, vocab_size, d_model, N, heads):  
        super().__init__()  
        self.N = N  
        self.embed = Embedder(vocab_size, d_model)  
        self.pe = PositionalEncoder(d_model)  
        self.layers = get_clones(EncoderLayer(d_model, heads), N)  
        self.norm = Norm(d_model)  
  
    def forward(self, src, mask):  
        x = self.embed(src)  
        x = self.pe(x)  
        for i in range(self.N):  
            x = self.layers[i](x, mask)  
        return self.norm(x)  
  
class Decoder(nn.Module):  
    def __init__(self, vocab_size, d_model, N, heads):  
        super().__init__()  
        self.N = N  
        self.embed = Embedder(vocab_size, d_model)  
        self.pe = PositionalEncoder(d_model)  
        self.layers = get_clones(DecoderLayer(d_model, heads), N)  
        self.norm = Norm(d_model)  
  
    def forward(self, trg, e_outputs, src_mask, trg_mask):  
        x = self.embed(trg)  
        x = self.pe(x)  
        for i in range(self.N):  
            x = self.layers[i](x, e_outputs, src_mask, trg_mask)  
        return self.norm(x)
```

```

class Transformer(nn.Module):
    def __init__(self, src_vocab, trg_vocab, d_model, N, heads):
        super().__init__()
        self.encoder = Encoder(src_vocab, d_model, N, heads)
        self.decoder = Decoder(trg_vocab, d_model, N, heads)
        self.out = nn.Linear(d_model, trg_vocab)

    def forward(self, src, trg, src_mask, trg_mask):
        e_outputs = self.encoder(src, src_mask)
        d_output = self.decoder(trg, e_outputs, src_mask, trg_mask)
        output = self.out(d_output)
        return output    # đầu ra là batch_size, seg_len_decoder, vocab_size decoder

```

```

import spacy
import re

# Tokenize
# khởi tạo 1 bộ tokenizer với ngôn ngữ được chọn sẵn
class tokenize(object):

    def __init__(self, lang):
        self.nlp = spacy.load(lang)

    def tokenizer(self, sentence):
        sentence = re.sub(
            r"[\*\"'\"'\n\\...\\+\\-\\/\\=\\(\\)‘•:\\.\\[\\]\\|’;]", " ", str(sentence))
        sentence = re.sub(r"[' ']+", " ", sentence)
        sentence = re.sub(r"!+", "!", sentence)
        sentence = re.sub(r",+", ",", sentence)
        sentence = re.sub(r"?+", "?", sentence)
        sentence = sentence.lower()
        return [tok.text for tok in self.nlp.tokenizer(sentence) if tok.text != " "]

```

```

import numpy as np

# tạo mask cho không nhìn thấy tương lai
# tại quá trình train của decoder ta feed song song các chuỗi các chuỗi kiểu 1,12,123,1234 với các nhãn là các từ của câu đó bắt đầu từ token thứ 2( sau
def nopeak_mask(size, opt):
    np_mask = np.triu(np.ones((1, size, size)),k=1).astype('uint8')
    # tạo ma trận tam giác trên với đường chéo chính là 0 và tất cả các phần tử phía trên là 1
    # 1 tượng trưng cho mask, điều này đồng nghĩa với việc che đi các từ sau từ hiện tại
    np_mask = (torch.from_numpy(np_mask) == 0)
    np_mask = np_mask.to(opt.device)
    return np_mask

# hàm trên tạo ra 1 ma trận mask sử dụng trước khi cho ma trận attention(q*transpose(k)) vào trong hàm softmax bằng cách lấy ma trận attention nhân với m

def create_masks(src, trg, opt):
    #src pad là id padding của bên encoder, câu lệnh dưới trả về false tại nơi là padding
    # src mask có chiều là batch_size * seq_len. sau unsqueeze(-2) thì sẽ có chiều là batch*1*seq_len
    src_mask = (src != opt.src_pad).unsqueeze(-2)

    if trg is not None:
        trg = trg.to(opt.device)
        trg_mask = (trg != opt.trg_pad).unsqueeze(-2).to(opt.device)
        size = trg.size(1) # get seq_len for matrix
        np_mask = nopeak_mask(size, opt)
        trg_mask = trg_mask & np_mask

    else:
        trg_mask = None
    return src_mask, trg_mask

# Sử dụng torch.utils.data thay vì torchtext.Legacy.data
import pandas as pd
from torch.utils.data import Dataset, DataLoader
import os
import pickle

```

```

class TranslationDataset(Dataset):
    """Custom Dataset cho dữ liệu dịch thuật"""
    def __init__(self, src_data, trg_data, src_field, trg_field): # yêu cầu các text của encoder và decoder tương ứng và các các xử lý của encoder và de
        self.src_data = src_data
        self.trg_data = trg_data
        self.src_field = src_field # quy định xử lý riêng của từng bên
        self.trg_field = trg_field

    def __len__(self):
        return len(self.src_data)

    def __getitem__(self, idx):
        src_tokens = self.src_field.tokenizer(self.src_data[idx])
        trg_tokens = self.trg_field.tokenizer(self.trg_data[idx])

        src_indices = [self.src_field.vocab.stoi.get(token, self.src_field.vocab.stoi['<unk>']) for token in src_tokens] # stoi là 1 dict, phương thức g
        trg_indices = [self.trg_field.vocab.stoi['<sos>']] + [self.trg_field.vocab.stoi.get(token, self.trg_field.vocab.stoi['<unk>']) for token in trg_
        # ở encoder thì không cần token sos và eos, bên decoder thì cần cả 2 để biết khi nào bắt đầu và kết thúc

        return {
            'src': src_indices,
            'trg': trg_indices
        }

```

```

def collate_fn(batch):
    """Pad batch sequences to the same length"""
    src_batch = [item['src'] for item in batch]
    trg_batch = [item['trg'] for item in batch]

    # Pad sequences
    max_src_len = max(len(seq) for seq in src_batch)
    max_trg_len = max(len(seq) for seq in trg_batch)

    src_padded = []
    trg_padded = []

    for src_seq, trg_seq in zip(src_batch, trg_batch):
        src_pad_idx = 0 # <pad> token index
        trg_pad_idx = 0

        src_padded.append(src_seq + [src_pad_idx] * (max_src_len - len(src_seq)))
        trg_padded.append(trg_seq + [trg_pad_idx] * (max_trg_len - len(trg_seq)))

    return torch.LongTensor(src_padded), torch.LongTensor(trg_padded)

def read_data(opt):
    if opt.src_data is not None:
        try:
            opt.src_data = open(opt.src_data, encoding = 'utf-8').read().strip().split('\n')
        except:
            print("error: '" + opt.src_data + "' file not found")
            quit()

    if opt.trg_data is not None:
        try:
            opt.trg_data = open(opt.trg_data, encoding = 'utf-8').read().strip().split('\n')
        except:
            print("error: '" + opt.trg_data + "' file not found")
            quit()

```

```

def create_fields(opt):
    spacy_langs = ['en', 'fr', 'de', 'es', 'pt', 'it', 'nl']
    src_lang = opt.src_lang[0:2] # Lấy ra 2 kí tự đầu tiên tượng trưng cho ngôn ngữ sử dụng của ta
    trg_lang = opt.trg_lang[0:2]
    if src_lang not in spacy_langs:
        print('invalid src language: ' + opt.src_lang + 'supported languages : ' + spacy_langs)
    if trg_lang not in spacy_langs:
        print('invalid trg language: ' + opt.trg_lang + 'supported languages : ' + spacy_langs)

    print("loading spacy tokenizers...")

    t_src = tokenize(opt.src_lang) # tạo bộ tokenizer của encoder
    t_trg = tokenize(opt.trg_lang)

    # Tạo vocabulary object đơn giản
    class Vocab:
        pass

```

```

# Tạo vocabulary object đơn giản
class Vocab:
    pass

class VocabField: # công cụ để build vocab
    def __init__(self, lang_tokenizer, special_tokens=['<pad>', '<sos>', '<eos>', '<unk>']):
        self.tokenizer = lang_tokenizer.tokenizer
        self.vocab = Vocab()
        self.special_tokens = special_tokens

    def build_vocab(self, sentences):
        word_freq = {}
        for sentence in sentences:
            tokens = self.tokenizer(sentence)
            for token in tokens:
                word_freq[token] = word_freq.get(token, 0) + 1

        # Tạo vocab
        self.vocab.stoi = {token: idx for idx, token in enumerate(self.special_tokens)} # tạo thuộc
        self.vocab.itos = {idx: token for token, idx in self.vocab.stoi.items()}

        idx = len(self.special_tokens)
        # gán id nhỏ cho các từ có tần số xuất hiện cao
        for word, freq in sorted(word_freq.items(), key=lambda x: x[1], reverse=True):
            self.vocab.stoi[word] = idx
            self.vocab.itos[idx] = word
            idx += 1

SRC = VocabField(t_src)
TRG = VocabField(t_trg)

return SRC, TRG

```



```

def create_dataset(opt, SRC, TRG):
    print("creating dataset and iterator... ")

    raw_data = {'src': [line for line in opt.src_data], 'trg': [line for line in opt.trg_data]}
    df = pd.DataFrame(raw_data, columns=["src", "trg"])
    df = df[(df['src'].str.count(' ') < opt.max_strlen) & (df['trg'].str.count(' ') < opt.max_strlen)]

    # Build vocabularies
    SRC.build_vocab(df['src'].tolist())
    TRG.build_vocab(df['trg'].tolist())

    opt.src_pad = SRC.vocab.stoi['<pad>']
    opt.trg_pad = TRG.vocab.stoi['<pad>']

    # Create dataset
    dataset = TranslationDataset(df['src'].tolist(), df['trg'].tolist(), SRC, TRG)

    # Create DataLoader
    train_iter = DataLoader(
        dataset,
        batch_size=opt.batchsize,
        shuffle=True,
        collate_fn=collate_fn
    )

    opt.train_len = len(train_iter) # số batch của 1 epoch

    return train_iter

def get_len(train):
    return len(train)

```

```

# Optimizer
class CosineWithRestarts(torch.optim.lr_scheduler._LRScheduler):
    """
    Cosine annealing with restarts.
    Parameters
    -----
    optimizer : torch.optim.Optimizer
    T_max : int
        The maximum number of iterations within the first cycle.
    eta_min : float, optional (default: 0)
        The minimum learning rate.
    last_epoch : int, optional (default: -1)
        The index of the last epoch.
    """

    def __init__(self,
                 optimizer: torch.optim.Optimizer,
                 T_max: int,
                 eta_min: float = 0.,
                 last_epoch: int = -1,
                 factor: float = 1.) -> None:
        # pylint: disable=invalid-name
        self.T_max = T_max
        self.eta_min = eta_min
        self.factor = factor
        self._last_restart: int = 0
        self._cycle_counter: int = 0
        self._cycle_factor: float = 1.
        self._updated_cycle_len: int = T_max
        self._initialized: bool = False
        super(CosineWithRestarts, self).__init__(optimizer, last_epoch)

```

```

def get_lr(self):
    """Get updated learning rate."""
    # HACK: We need to check if this is the first time get_lr() was called, since
    # we want to start with step = 0, but _LRScheduler calls get_lr with
    # last_epoch + 1 when initialized.
    if not self._initialized:
        self._initialized = True
        return self.base_lrs

    step = self.last_epoch + 1
    self._cycle_counter = step - self._last_restart

    lrs = [
        (
            self.eta_min + ((lr - self.eta_min) / 2) *
            (
                np.cos(
                    np.pi *
                    ((self._cycle_counter) % self._updated_cycle_len) /
                    self._updated_cycle_len
                ) + 1
            )
        ) for lr in self.base_lrs
    ]

    if self._cycle_counter % self._updated_cycle_len == 0:
        # Adjust the cycle length.
        self._cycle_factor *= self.factor
        self._cycle_counter = 0
        self._updated_cycle_len = int(self._cycle_factor * self.T_max)
        self._last_restart = step

    return lrs

```

```

from tqdm import tqdm

class Opt():
    pass

en_dir = "data/english.txt"
fr_dir = "data/french.txt"
# Thay vì dùng đường dẫn C:\Users\..., hãy dùng:
def train_model(model, opt, loader, optimizer, scheduler, criterion):
    model.train()
    for i in range(opt.epochs):
        losses = 0
        for src_padded, trg_padded in tqdm(loader):
            optimizer.zero_grad()
            src_padded, trg_padded = src_padded.to(opt.device), trg_padded.to(opt.device)
            nopeak_trg = nopeak_mask(trg_padded.size(-1), opt)
            src_mask, trg_mask = create_masks(src_padded, trg_padded[:, :-1], opt)
            output = model(src_padded, trg_padded[:, :-1], src_mask, trg_mask) # đầu vào của decoder không bao gồm eos
            targets = trg_padded[:, 1:]
            loss = criterion(output.reshape(-1, output.size(-1)), targets.reshape(-1))
            loss.backward()
            losses += loss.item()
            optimizer.step()
            scheduler.step()
        print(f'Losses of Epoch {i}: {losses/len(loader)}')

def main():
    opt = Opt()
    opt.src_data = en_dir
    opt.trg_data = fr_dir
    opt.src_lang = "en_core_web_sm"
    opt.trg_lang = 'fr_core_news_sm'
    opt.epochs = 5
    opt.d_model = 512
    opt.n_layers = 2
    opt.heads = 8
    opt.dropout = 0.1
    opt.batchsize = 128
    opt.printevery = 32
    opt.lr = 0.0001
    opt.device = device
    opt.max_strlen = 100

    read_data(opt)
    src_vocab_field, trg_vocab_field = create_fields(opt)
    loader = create_dataset(opt, src_vocab_field, trg_vocab_field)
    model = Transformer(len(src_vocab_field.vocab.itos), len(trg_vocab_field.vocab.itos), opt.d_model, opt.n_layers, opt.heads).to(opt.device)
    optimizer = torch.optim.Adam(model.parameters(), lr = opt.lr)
    scheduler = CosineWithRestarts(optimizer, T_max = 5, eta_min = 1e-6)
    criterion = nn.CrossEntropyLoss(ignore_index = 0) # ignore padd
    train_model(model, opt, loader, optimizer, scheduler, criterion)

main()

loading spacy tokenizers...
creating dataset and iterator...
6%|██████| | 71/1211 [03:44<55:35, 2.93s/it]

```